EXPERIMENT 2: CLI, GUI, VUI

Name: Kaajal G Roll number: 240701220

Aim:

The aim of this experiment is to compare three modes of human–computer interaction: Command-Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI). The experiment seeks to evaluate their efficiency, usability, and accessibility by performing similar tasks across each interface, highlighting their strengths, limitations, and suitability for different user contexts.

1. CLI

Student_grader.py

```python
students = {}

def assign_grade(marks):
    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B+"
    elif marks >= 60:
        return "B"
    else:
        return "C"


def add_student():
    name = input("Enter student name: ")
    marks = int(input("Enter marks: "))

    grade = assign_grade(marks)
    students[name] = {"marks": marks, "grade": grade}

    print(f"Student {name} added with Grade {grade}")


def view_students():
    if students:
        print("\nStudent List")
        for name, data in students.items():
            print(f"Name: {name} | Marks: {data['marks']} | Grade: {data['grade']}")
    else:
        print("No students added.")


def class_average():
    if students:
```

EXPERIMENT 2: CLI, GUI, VUI

```python
        total = sum(data["marks"] for data in students.values())
        avg = total / len(students)
        print("Class Average:", avg)
    else:
        print("No data available.")


def main():
    while True:
        print("\n1. Add Student")
        print("2. View Students")
        print("3. Class Average")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_student()

        elif choice == "2":
            view_students()

        elif choice == "3":
            class_average()

        elif choice == "4":
            print("Exiting program")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()

```

Output:

A. Add Student

```
1. Add Student
2. View Students
3. Class Average
4. Exit
Enter your choice: 1
Enter student name: Kaajal G
Enter marks: 99
Student Kaajal G added with Grade A+

1. Add Student
2. View Students
3. Class Average
4. Exit
Enter your choice: 1
Enter student name: Hema k
Enter marks: 87
Student Hema k added with Grade A
```

EXPERIMENT 2: CLI, GUI, VUI

B. View Students

```
1. Add Student
2. View Students
3. Class Average
4. Exit
Enter your choice: 2

Student List
Name: Kaajal G | Marks: 99 | Grade: A+
Name: Hema k | Marks: 87 | Grade: A
Name: Gandhi S | Marks: 98 | Grade: A+
Name: Nihaal G | Marks: 67 | Grade: B
Name: Geetha T | Marks: 86 | Grade: A
Name: Vishal G | Marks: 79 | Grade: B+
```

C. Class Average

```
1. Add Student
2. View Students
3. Class Average
4. Exit
Enter your choice: 3
Class Average: 86.0
```

D. Exit

```
1. Add Student
2. View Students
3. Class Average
4. Exit
Enter your choice: 4
Exiting program
```

2. GUI

et.py

```python
import tkinter as tk

expenses = []

def add_expense():
    item = item_entry.get()
    amount = amount_entry.get()

    if item != "" and amount != "":
        listbox.insert(tk.END, f"{item} - ₹{amount}")
        expenses.append(float(amount))

        item_entry.delete(0, tk.END)
        amount_entry.delete(0, tk.END)

```

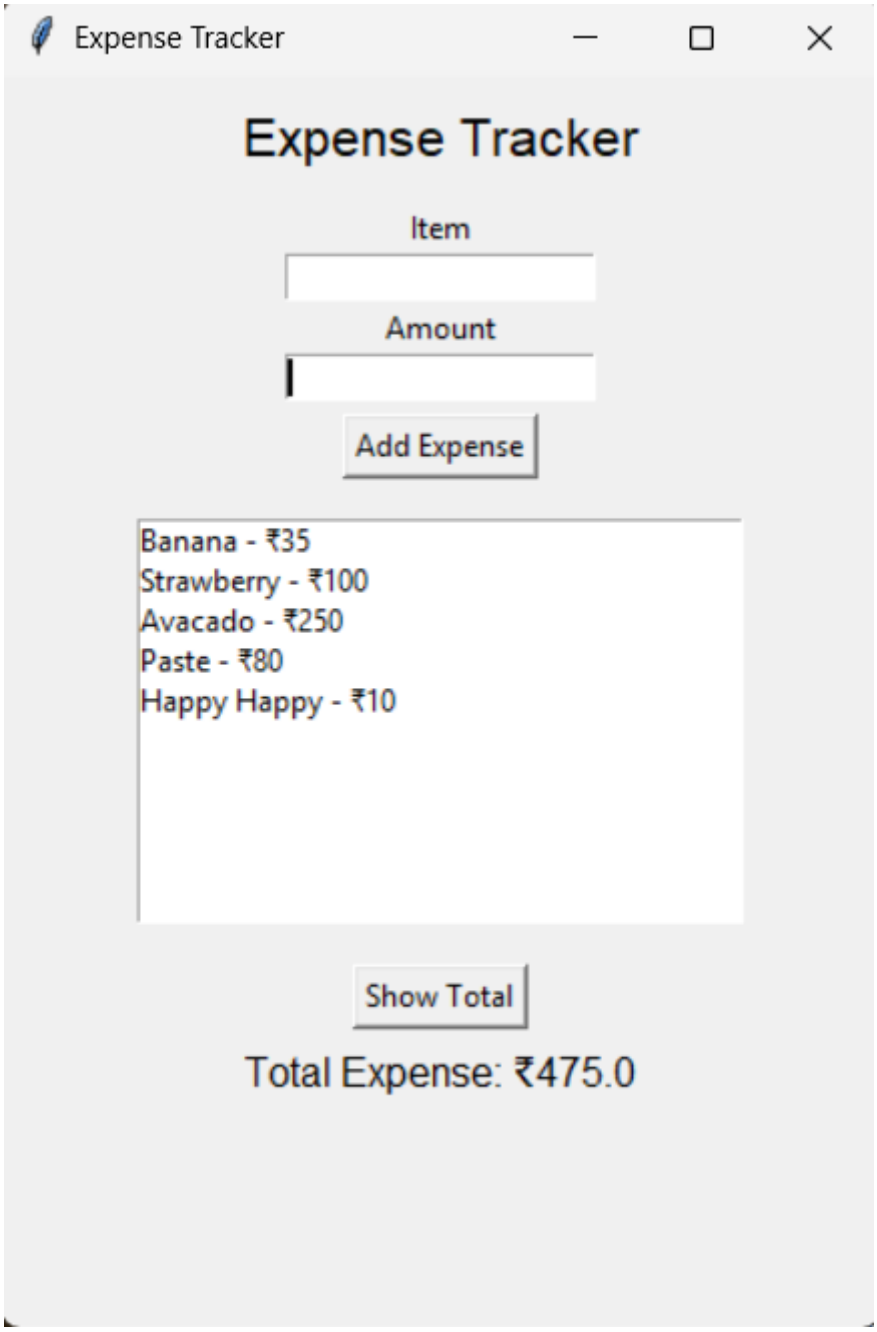
EXPERIMENT 2: CLI, GUI, VUI

```python
def calculate_total():
    total = sum(expenses)
    total_label.config(text=f"Total Expense: ₹{total}")

root = tk.Tk()
root.title("Expense Tracker")
root.geometry("350x500")

title = tk.Label(root, text="Expense Tracker", font=("Arial", 16))
title.pack(pady=10)

item_label = tk.Label(root, text="Item")
item_label.pack()

item_entry = tk.Entry(root)
item_entry.pack()

amount_label = tk.Label(root, text="Amount")
amount_label.pack()

amount_entry = tk.Entry(root)
```

```python
amount_entry.pack()

add_button = tk.Button(root, text="Add Expense", command=add_expense)
add_button.pack(pady=5)

listbox = tk.Listbox(root, width=40)
listbox.pack(pady=10)

total_button = tk.Button(root, text="Show Total", command=calculate_total)
total_button.pack(pady=5)

total_label = tk.Label(root, text="Total Expense: ₹0", font=("Arial", 12))
total_label.pack()

root.mainloop()
```

EXPERIMENT 2: CLI, GUI, VUI

Output:



3. VUI

vui.py

```
import pyttsx3

tasks = []

engine = pyttsx3.init()

def speak(text):
    print(text)
    engine.say(text)
    engine.runAndWait()
```

EXPERIMENT 2: CLI, GUI, VUI

```python
def add_task(task):
    tasks.append(task)
    speak(f"Task {task} added")

def view_tasks():
    if tasks:
        speak("Your tasks are")
        for i, task in enumerate(tasks, 1):
            speak(f"{i}. {task}")
    else:
        speak("No tasks to show")

def remove_task(task_number):
    if 0 < task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        speak(f"Task {removed_task} removed")
    else:
        speak("Invalid task number")

# Simulated voice input
def recognize_speech():
    command = input("Type command (simulate voice): ")
    return command.lower()

def main():
    while True:
        speak("Say add task, view tasks, remove task, or exit")

        command = recognize_speech()

        if not command:
            continue

        if "add task" in command:
            speak("What is the task")
            task = recognize_speech()
            if task:
```

```python
                add_task(task)

        elif "view tasks" in command:
            view_tasks()

        elif "remove task" in command:
            speak("Which task number")
            num = recognize_speech()
            if num.isdigit():
                remove_task(int(num))

        elif "exit" in command:
            speak("Exiting program")
            break

        else:
            speak("Invalid command")

if __name__ == "__main__":
    main()
```

EXPERIMENT 2: CLI, GUI, VUI

Output:

```
Say add task, view tasks, remove task, or exit
Type command (simulate voice): add task
What is the task
Type command (simulate voice): finish your food
Task finish your food added
Say add task, view tasks, remove task, or exit
Type command (simulate voice): view tasks
Your tasks are
1. finish your food
Say add task, view tasks, remove task, or exit
Type command (simulate voice): exit
Exiting program
```

Result:

The experiment successfully demonstrated how a single notes management application can be implemented across three interaction paradigms — **CLI, GUI, and VUI**. Each interface provided unique insights into usability and accessibility: the CLI offered simplicity and direct control, the GUI enhanced user experience through visual interaction, and the VUI introduced hands-free convenience with natural language commands.